ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
DEPARTMENT OF INFORMATICS
Algorithms, Spring Semester 2026
Programming Assignment
ATHANASIOS GOURDOMICHALIS

Introduction

A dietitian is attempting to design a customized meal plan for a client. Specifically, the client has requested a high-protein diet for one day of the week (e.g., every Sunday). The dietitian has access to the following dataset:

- A parameter $\boldsymbol{W}$ representing an upper bound on the total caloric intake allowed for Sunday's meal plan.
- A set of $\boldsymbol{n}$ distinct food portion options. The term "portion" here refers either to a single unit of a specific item or a defined quantity of a certain type of food (e.g., 45 g of rolled oats, one package of potato chips, one pork gyros wrap, one avocado, etc.). For each portion $\boldsymbol{i}$, there are two integer parameters: its caloric value, denoted as $\boldsymbol{c_i}$, and its protein content, denoted as $\boldsymbol{p_i}$. We assume that for every $\boldsymbol{i}$, $\boldsymbol{c_i} \leq \boldsymbol{W}$ holds. Furthermore, each portion can be included at most once in the designed meal plan (i.e., portions are indivisible; they must either be selected entirely or excluded completely).

The dietitian's objective is to construct a meal plan that maximizes total protein content subject to the constraint that the cumulative calories do not exceed $\boldsymbol{W}$. Your goal is to assist the dietitian by implementing and experimentally evaluating the practical performance of three algorithms.

1. Two Dynamic Programming Algorithms

The first two algorithms are based on the dynamic programming technique.

- **Algorithm 1:** Implement the dynamic programming approach covered in the lecture notes (Slides 35-36, Unit 10) to achieve a time complexity of $\boldsymbol{O(nW)}$.
- **Algorithm 2:** Design an alternative dynamic programming algorithm whose running time depends on the parameters $\boldsymbol{n}$ and $\boldsymbol{p_{\max}}$, where $\boldsymbol{p_{\max}} = \max_{i=1\ldots n} \boldsymbol{p_i}$.

Hint for Algorithm 2: Let the distinct food portions be indexed from 1 to $\boldsymbol{n}$. Define $\boldsymbol{X[k,p]}$ as the minimum caloric capacity required to achieve a total protein content of at least $\boldsymbol{p}$, utilizing a subset of the first $\boldsymbol{k}$ portions as input. Consider how to leverage this recurrence relation to design your algorithm.

2. A Greedy Algorithm

The dietitian realizes that the previous dynamic programming algorithms are somewhat complex to explain directly to the client. Consequently, he decides to evaluate a third, more intuitive approach that he devised himself.

Algorithm 3 is a greedy algorithm operating as follows:

- First, sort the portions in non-increasing order of their efficiency ratios, $\boldsymbol{p_i/c_i}$, and re-index the portions from $\boldsymbol{1}$ to $\boldsymbol{n}$ such that:

$$\frac{\boldsymbol{p_1}}{\boldsymbol{c_1}} \geq \frac{\boldsymbol{p_2}}{\boldsymbol{c_2}} \geq \cdots \geq \frac{\boldsymbol{p_n}}{\boldsymbol{c_n}}$$

- Iteratively append portions to the solution starting from portion 1, then portion 2, and so on, until encountering a portion that, if added, would violate the strict caloric upper bound $\boldsymbol{W}$ (the algorithm terminates at the portion preceding the violating one).
- The algorithm returns the best solution between: **(a)** the greedy solution constructed in the previous steps, and **(b)** a solution containing exclusively the single portion that features the highest individual protein content.

3. Assignment Requirements

(i) (15 points) Explain the design logic behind Algorithm 2 and formalize it using pseudocode. Provide a comprehensive asymptotic complexity analysis.

(ii) (30 points) Implement Algorithms 1–3 in either Java or Python. If time constraints prevent the implementation of all three, implement at least Algorithm 1 and Algorithm 3 to enable comparative evaluation. Every algorithm must accept as input: the integer parameter $\boldsymbol{W}$, an integer array Calories (or an equivalent linear data structure) containing the caloric values of each portion, and a corresponding integer array Protein containing the protein content of each portion.

The output must be the total protein content achieved by the respective meal plan. For validation purposes (sanity check), print the cumulative caloric cost of the generated meal plan to verify feasibility. You are not required to output the specific subset of items selected for the meal plan.

(iii) (55 points) This component involves the experimental evaluation of the three algorithms. Select at least 5 distinct values for $\boldsymbol{n}$, and generate 10 random problem instances for each chosen value of $\boldsymbol{n}$ (resulting in a total execution over at least 50 test instances). We aim to evaluate algorithmic performance on relatively large input sizes; thus, you may indicatively use $\boldsymbol{n} = 20, 50, 100, 500, 1000$ (these values are not mandatory, and you are encouraged to experiment with even larger scales).

Subsequently, compute and report the following metrics:

- For each algorithm and each input size $\boldsymbol{n}$, calculate the average execution time in milliseconds (**ms**) averaged across the 10 random instances. You may visualize these metrics using a chart plot displaying the execution times of all three algorithms as a function of $\boldsymbol{n}$, or tabulate the mean execution times.

- **Evaluation of Algorithm 3:** Unlike Algorithms 1 and 2, which are exact algorithms guaranteed to find an optimal solution, Algorithm 3 is a heuristic without such absolute guarantees. It will consistently yield a feasible solution, but its approximation accuracy must be empirically quantified. Compute the average approximation ratio for each value of $\boldsymbol{n}$.

For a given problem instance $\boldsymbol{I}$, the approximation ratio of Algorithm 3 is defined as $\boldsymbol{SOL(I)/OPT(I)}$, where $\boldsymbol{OPT(I)}$ is the optimal protein capacity (computed by the exact algorithms) and $\boldsymbol{SOL(I)}$ is the protein capacity produced by Algorithm 3. Record the mean approximation ratio across the 10 instances for each value of $\boldsymbol{n}$.

Conclude your report by discussing the behavior of the three algorithms, focusing primarily on their time and space complexities. For Algorithm 3, analyze the trade-off between execution speed and the approximation ratio relative to the optimal solution. Incorporate any additional insights derived from the experimental benchmarks (e.g., whether one algorithm consistently outperforms the others or if specific problem structures favor a particular approach).

4. Implementation Guidelines

Random Instance Generation: Once a value for $\boldsymbol{n}$ is selected, generate a random instance consisting of n portions by producing random integers for the parameters $\boldsymbol{W, c_1, \ldots, c_n, p_1, \ldots, p_n}$ using uniform pseudorandom number generators within appropriate integer intervals. Exercise care to avoid generating trivial instances that are easily solved optimally by all approaches. Consider the following structural guidelines:

- Ensure that $\boldsymbol{c_i \leq W}$ holds for all $\boldsymbol{i}$.

- If $\boldsymbol{\sum_{i=1}^{n} c_i \leq W}$, the problem becomes trivial as the optimal choice is to select all items. To ensure non-triviality, $\boldsymbol{W}$ should be selected uniformly at random from a sub-interval such as $\boldsymbol{[0.4 \cdot \sum_{i=1}^{n} c_i , 0.7 \cdot \sum_{i=1}^{n} c_i]}$. You may vary this range based on preliminary empirical testing.

- Since the running time of Algorithm 1 depends heavily on $\boldsymbol{W}$ and Algorithm 2 depends on $\boldsymbol{p_{max}}$, ensure these values are sufficiently large and scalable with respect to n to allow for a meaningful algorithmic comparison.

  Based on the comments, you must ultimately decide how to handle the generation of random instances. There is no single correct answer here. On the contrary, there are many options that are satisfactory for conducting a reliable comparison of the algorithms. Additionally, you may use two different methods for generating random instances if you wish (e.g., 5 instances using the first method and 5 using the other, or for greater reliability, 10 instances with each method).

PROJECT FILES

- **Source code:** Place the files you have created, along with any other source code files required for your assignment to compile, into a directory named src. Your code must be free of syntax errors and fully compilable. The grading of the assignments concerns the evaluation of compilable implementations and not debugging compilation errors. Furthermore, ensure that you add explanatory comments wherever you deem necessary within your code.

- **Random instances:** Place the random instances you generated into a directory named data. You may save each instance as a .txt file or in any other format that serves your implementation.

- **Report file:** A document in .pdf format containing your answers to the questions posed in sections **(i)** and **(iii).** For question **(iii),** you must also include an explanation of the methodology used to generate the random instances. Finally, it is highly recommended to explain how your algorithms are executed (e.g., via the command line or within an IDE, and how the input data is provided).